Lecture 5
Merging Branches and Resolving Conflicts

Fast-forward merges, merge commits, conflict markers, and `git merge --abort`

Learning goals

- Merge one branch into another and say which kind of merge Git performed.
- Explain why a merge commit has two parents, and how Git combines two lines of work.
- Read the conflict markers in a file and resolve a conflict step by step.
- Cancel a merge safely with `git merge --abort`.
- Delete branches once they are merged.

1 What merging means

In Lecture 4 you created branches and let the history fork. **Merging** is the other half of the story: bringing the commits of one branch into another, so that the finished work ends up on `main`.

The command is `git merge`, and its direction matters:

Key idea

You first *switch to the branch that should receive the work*, then run `git merge <other-branch>`. The current branch changes; the other branch is left untouched. `git merge waffles` while on `main` means "bring `waffles` into `main`".

Before every merge, get into the habit of two checks: `git status` (the working directory should be clean, with everything committed) and `git branch --show-current` (you must be on the receiving branch).

For this lecture we use a practice repository with the same starting point on every computer. From the course folder run:

```
$ bash codeHints/L05/setup_demo5.sh
$ cd practice/demo5
$ git log --oneline --graph --all --decorate
* fbed2d3 (vanilla) Pancakes with vanilla
| * 0725dd2 (syrup) Pancakes with syrup
|/
| * 633affa (shopping) Add shopping list
|/
| * df478ec (crepes) Add crepe recipe
|/
* 7ad9635 (HEAD -> main) Add pancake recipe
* 3f587ea Add README
```

`main` has two commits, and four branches start from its tip. We will merge them one after another, and each one teaches a different case. (To restart at any time: leave the folder with `cd ../..`, run `setup_demo5.sh --reset`, then `cd practice/demo5`.)

2 Case 1: the fast-forward merge

The branch `crepes` added one commit on top of `main`'s latest commit, and `main` has not moved since. Nothing needs combining: Git only has to slide the `main` label forward.

```
$ git merge crepes
Updating 7ad9635..df478ec
Fast-forward
 crepes.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 crepes.txt
```

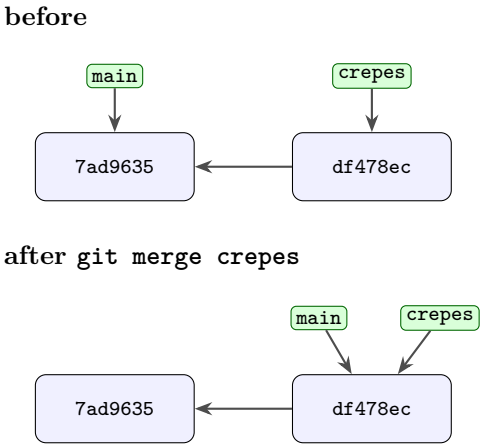


Figure 1: A fast-forward merge. No new commit is created: the `main` label just moves forward to the commit `crepes` already points at. The history stays a straight line.

Quick fact

A fast-forward is possible only when the receiving branch has *no commits of its own* beyond the shared starting point. If you ever want a visible merge commit even then, use `git merge --no-ff`.

3 Case 2: the merge commit

Next, `shopping`. It also started from the old `Add pancake recipe` commit, but `main` has moved on (it now has the crepe commit). The two lines of work have diverged, so Git cannot just slide a label. It must *combine* them:

```
$ git merge --no-edit shopping
Merge made by the 'ort' strategy.
 shopping.txt | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 shopping.txt
```

(The word `ort` is just the name of Git's merging method; older versions print `recursive`. Ignore it. Without `--no-edit`, Git opens a text editor so that you can confirm the merge message: save and close it, which in `vim` means pressing `Esc`, typing `:wq` and Enter. We skip the editor with `--no-edit`.)

Git made a **merge commit**, a special commit with *two parents* (two commits directly before it): one from each line of work.

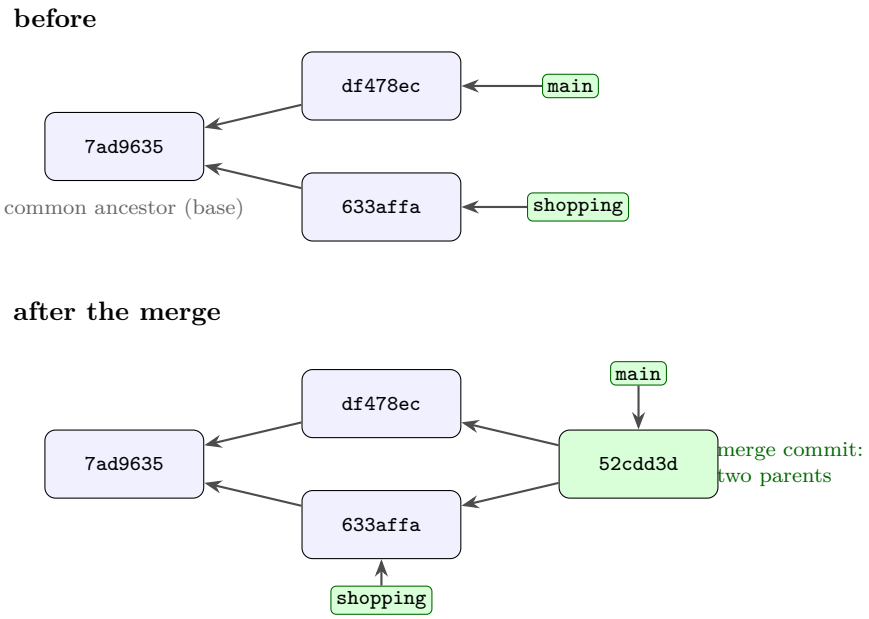


Figure 2: A three-way merge. Git finds the common ancestor of the two branches and creates a merge commit whose two parents are the two tips. `main` moves onto it; `shopping` stays where it was.

The name *three-way* comes from the three versions Git looks at: the common ancestor (the **base**) and the two branch tips. It compares each tip with the base to see what each side changed, then combines the changes. The rules are simple:

What happened since the base	Result
only one side changed a part of a file	Git takes that side's change
both sides changed *different, non-adjacent* parts of a file	Git applies both changes
both sides changed the *same part* differently	**conflict**: Git asks you

Look at the history, and at the two parents of the merge commit:

```
$ git log --oneline --graph --decorate -4
*   52cdd3d (HEAD -> main) Merge branch 'shopping'
|\
| * 633affa (shopping) Add shopping list
* | df478ec (crepes) Add crepe recipe
|/
* 7ad9635 Add pancake recipe
$ git log -1 --format="%h parents: %p"
52cdd3d parents: df478ec 633affa
$ git log --merges --oneline
52cdd3d Merge branch 'shopping'
```

(The ID of any merge commit depends on the time you run the merge, so yours will differ from `52cdd3d`. The IDs of the branch commits are fixed.) The command `git log --merges` lists only merge commits.

One more merge before the conflict. Merge `syrup` now (the next case needs it). It changes line 1 of `recipes.txt`, and nothing on `main` has touched that file, so Git applies the change on its own and makes a second merge commit:

```
$ git merge --no-edit syrup
Merge made by the 'ort' strategy.
 recipes.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ cat recipes.txt
Pancakes: flour, milk, eggs, syrup
```

4 Case 3: the conflict

Now the branch we have been waiting for. `main` (through `syrup`) and `vanilla` both changed the *same line* of `recipes.txt`. Git cannot know which version you want:

```
$ git merge vanilla
Auto-merging recipes.txt
CONFLICT (content): Merge conflict in recipes.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Key idea

A conflict is not an error and nothing is broken. It is Git saying: "Both sides edited this spot. A human has to decide." The merge is paused halfway, waiting for you. Your earlier commits are completely safe.

4.1 Step 1: see what is going on

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   recipes.txt

no changes added to commit (use "git add" and/or "git commit -a")
$ git status --short
UU recipes.txt
```

"Unmerged paths" means "files that are still conflicted". `both modified` (`UU` in the short form) marks the conflicted file. Git has already merged everything it could. If several files conflict you will see several lines here; resolve each file in turn, `git add` each one, and make one `git commit` at the end. Note the last hint: `git merge --abort` is your escape hatch (Step 3).

4.2 Step 2: read the conflict markers

Open the file in any text editor (for example `nano recipes.txt`, or your usual editor), or just print it with `cat`. Git has written both versions into it, separated by markers:

```
$ cat recipes.txt
<<<<<<< HEAD
Pancakes: flour, milk, eggs, syrup
=======
Pancakes: flour, milk, eggs, vanilla
>>>>>>> vanilla
```

`<<<<<<< HEAD`	start marker. Below it: **your side**, the branch you are on (`HEAD` = `main`)
`Pancakes: flour, milk, eggs, syrup`	your version of the line
`=======`	divider between the two versions
`Pancakes: flour, milk, eggs, vanilla`	the **other side**, from the branch being merged
`>>>>>>> vanilla`	end marker, with the branch name

Figure 3: Anatomy of a conflict in `recipes.txt`. The three marker lines are not part of your text; they must all be removed.

4.3 Step 3: practise the escape hatch

Before changing anything, try the safety net. `git merge --abort` cancels the whole merge, and Git puts everything back as it was before you typed `git merge` (provided the working directory was clean then, as we required):

```
$ git merge --abort
$ git status
On branch main
nothing to commit, working tree clean
$ git merge vanilla                  # the same conflict appears again
```

This is the safest way to practise: start a merge, look at the conflict, abort, and try again. `--abort` works only while the merge is unfinished; after the final `git commit` the merge is done.

4.4 Step 4: decide and edit

Open `recipes.txt` in your editor. You have three choices: keep your side, keep the other side, or (the usual choice) combine them by writing the line you actually want. Delete the marker lines and leave only the final text. We want both ingredients:

```
Pancakes: flour, milk, eggs, syrup, vanilla
```

Pitfall

The file must contain *no* `<<<<<<<`, `=======` or `>>>>>>>` lines when you are finished. Git will not stop you from committing them, and the file will then be broken. Search for `<<<<` before you continue.

4.5 Step 5: tell Git it is resolved, and finish

```
$ git add recipes.txt
$ git status
On branch main
All conflicts fixed but you are still merging.
```

```
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   recipes.txt

$ git commit --no-edit
[main 6e9ef86] Merge branch 'vanilla'
```

`git add` here means "this file is resolved". `git commit` completes the merge with the prepared message `Merge branch 'vanilla'` (drop `--no-edit` to edit it; if an editor opens, save and close it). Your ID will differ from `6e9ef86`.

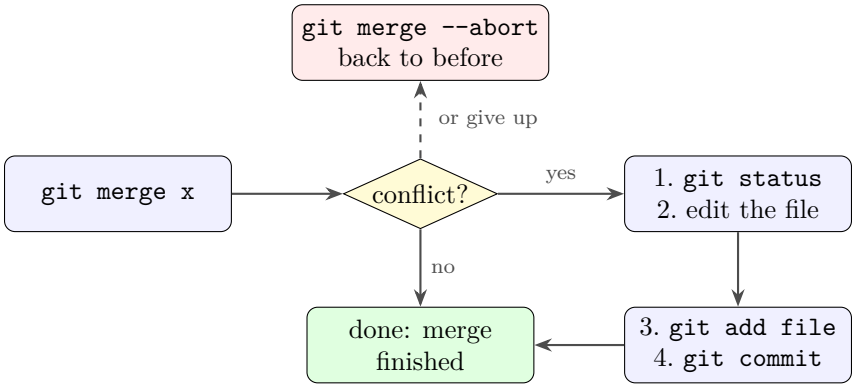


Figure 4: The conflict workflow. You can leave through the red box at any time before the final `git commit`.

Quick fact

Git conflicts not only when two people change the *same* line, but also when changes touch *directly adjacent* lines. If `main` had added a new line directly under the `Pancakes` line, then merging `syrup` (which edits that line) would have conflicted too. Conflicts are normal in teamwork, and they are easy once you have resolved a few.

Quick fact

The other direction. You can also merge `main` *into* a feature branch, to bring it up to date: `git switch feature`, then `git merge main`. The feature branch is the receiver and `main` is unchanged. If you merge in the wrong direction by mistake, do not panic: a finished merge can be undone with `git revert -m 1 <merge-commit>`. (A merge commit has two parents, so Git needs to know which one to keep: `-m 1` means the first parent, the branch you were on. For a merge you have *not* pushed, Lecture 10 shows `ORIG_HEAD`.)

5 Cleaning up

A merged branch has done its job. Its commits are now part of `main`, so the label can go. In Lecture 4 the command `git branch -d` refused to delete an unmerged branch; now it agrees:

```
$ git branch -d crepes shopping syrup vanilla
Deleted branch crepes (was df478ec).
Deleted branch shopping (was 633affa).
Deleted branch syrup (was 0725dd2).
Deleted branch vanilla (was fbed2d3).
$ git branch
* main
```

Deleting only removes the label; the commits stay in `main`'s history. The usual life cycle of a branch is therefore: *create, commit, merge, delete.*

Run `git log --oneline --graph` now: the graph tells the whole story, with three merge commits, each with two parents.

Pitfall

Two common merge mistakes. (1) Merging in the wrong direction: always `git switch` to the receiving branch first, and read the first line of `git status`. (2) Starting a merge with uncommitted changes: commit or discard them first, otherwise Git may refuse or mix them into the merge.

Quick fact

Merging a branch that is already included gives `Already up to date.` and changes nothing. (For regretting a finished merge, see the note on `git revert -m 1` above.)

6 Homework 5

Homework 5: Merge them all

From the course folder run:

```
$ bash codeHints/L05/setup_hw5.sh
$ cd practice/hw5
```

You get `main` (two commits) and four branches: `drinks`, `desserts`, `spicy` and `vegan`. Look first with `git log --oneline --graph --all --decorate`. Do the tasks *in this order*: the branch you merge first stands above the `=======` divider in a later conflict, and the questions in 5.6 depend on that. To start over, run `setup_hw5.sh --reset` from the course folder.

5.1 First merge

Merge `drinks` into `main`. (Which kind of merge will it be? Check that Git printed `Fast-forward`.)

5.2 Second merge

Merge `desserts`. Use `--no-edit`.

5.3 Third merge

Merge `spicy`. Use `--no-edit`.

5.4 The conflict

Merge `vegan`. Git reports a conflict in `menu.txt`. First look: run `git status`, open the file, then cancel with `git merge --abort` and check that `git status` is clean. Now merge `vegan` again and resolve it for real: line 1 of `menu.txt` must mention *both* `spicy` and `vegan` (for example `Pizza (spicy, vegan)`), line 2 stays `Pasta`, and no marker lines may remain. Add the file and finish with `git commit --no-edit`.

5.5 Clean up

Delete the four merged branches with `git branch -d`. Only `main` should remain.

5.6 Questions

In the file `practice/hw5_answers.txt` (next to the `hw5` folder, outside the repository; create it with `printf` or `echo` in Git Bash, not with a PowerShell redirect) write two lines:

```
above=<spicy or vegan>
merge-commits=<number>
```

`above` is the branch whose version of the line was *between* `<<<<<<< HEAD` and `=======` during the conflict in 5.4. `merge-commits` is the number of merge commits in `main` (look at `git log --merges --oneline`).

Check yourself: from the course folder run `bash codeHints/L05/check_hw5.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L05/`).

7 Summary

- `git merge X` brings branch `X` into the branch you are on. Switch to the receiver first; start from a clean working directory.
- **Fast-forward**: the receiver has nothing new, so its label just slides forward. **Merge commit**: both sides have new commits; Git combines them in a commit with two parents.
- A **conflict** means both sides changed the same spot. Open the file, choose or combine the versions, remove the markers, `git add`, then `git commit`. Use `git merge --abort` to cancel.
- Afterwards, delete the branch with `git branch -d`.

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 5

```
$ bash codeHints/L05/setup_hw5.sh
$ cd practice/hw5
$ git merge drinks
Fast-forward
$ git merge --no-edit desserts
$ git merge --no-edit spicy
$ git merge vegan                    # conflict in menu.txt
$ git status                         # look, then cancel:
$ git merge --abort
$ git merge vegan                    # conflict again, now resolve it
$ cat menu.txt
<<<<<<< HEAD
Pizza (spicy)
=======
Pizza (vegan)
>>>>>>> vegan
Pasta
```

Edit `menu.txt` so that it contains exactly

```
Pizza (spicy, vegan)
Pasta
```

then finish and clean up:

```
$ git add menu.txt
$ git commit --no-edit
$ git branch -d drinks desserts spicy vegan
$ git log --merges --oneline         # three lines
$ printf 'above=spicy\nmerge-commits=3\n' > ../hw5_answers.txt
```

5.1. `main` had no commits of its own beyond the shared start, so Git fast-forwarded: no merge commit, and `main` now points where `drinks` points. If you used `--no-ff`, you get an extra merge commit and the checker reports four merges.

5.2 and 5.3. `main` had moved (it contains `drinks`), so Git made a merge commit each time. `desserts` adds a new file and `spicy` edits `menu.txt`, which `main` has not changed, so there is nothing to resolve.

5.4. `spicy` and `vegan` both changed line 1 of `menu.txt`, which makes a conflict. Between `<<<<<<< HEAD` and `=======` stands *your side*, the branch you are on. Because `spicy` was merged into `main` first, that is the spicy version, which is why the answer to 5.6 is `above=spicy`. After the abort `git status` says `working tree clean`: the merge never happened, and you can try again. Any text that mentions both words on line 1 passes the checker. The line `Pasta` was not part of the conflict and stays unchanged. Check that no marker line is left with `grep -n '<<<<\|====\|>>>>' menu.txt` (it should print nothing).

5.5. `git branch -d` works because every branch is now fully merged. Had Git refused, that would

indicate an unmerged branch.

5.6. Three merge commits: `desserts`, `spicy` and `vegan`. The `drinks` fast-forward made none.

Common mistakes.

- Merging `vegan` before `spicy`. Then `vegan`'s version stands above the divider. The checker notices the wrong order (check "spicy was merged BEFORE vegan") and expects `above=spicy`, so follow the order of the tasks.
- Committing with the marker lines still in the file. Open the file and check.
- Merging while on the wrong branch (for example on `drinks`). Check with `git branch --show-current` before every merge.
- Running `git commit` without `git add menu.txt` first: Git refuses ("Committing is not possible because you have unmerged files").
- Running `git merge --abort` *after* the final commit: it no longer applies. Use `setup_hw5.sh --reset` and start again.